

```
<meta name="theme-color"
content="#3367D6" />
<meta name="viewport"
content="width=device-width, initial-
scale=1" />
```



Note: One of the interesting things in the above *manifest.json* file is the use of the long links for the icon's *src* property, like 'src': *https://cdn.glitch.com/a812b85a-f21e-4582-a22f-7d2a24581d91%2F512.png?v=1583924262338*. This is unique to Glitch projects. In other typical projects you are expected to upload your appropriately sized icon files into the */icons* folder under the */public* folder. However, Glitch treats every resource that you upload as an asset and stores it in a Content Delivery Network (CDN) path that is accessible from everywhere. Hence the *src* which normally would be */public/icons/img192.png* is now replaced by the icon file's equivalent CDN path, which you can get from the object's property when you click and select the item that you have uploaded into Assets.

Showing a custom offline page

One of the basic requirements for a PWA is to be able to show a custom offline page when the user is not connected to the network, so that the offline page of Chrome does not show. This is achieved in steps, first by adding a custom offline page to our project's public files and then by setting up service workers to cache and use those files for a good offline experience.

So, we start our work by adding an *offline.html* to our public folder which has only the following changes to start with from the working *index.html*. Change the item under ** counts to display something like in the code section below.

```
<section class="counts">
  <ul id="counts">
    <em>counts unavailable as you
```

```
are offline</em>
  </ul>
</section>
</main>
```

Now we put in some boilerplate code in *public/ service-worker.js* to perform the following tasks, as explained in the code of *service-worker.js* below, namely:

1. Create a local cache and add *offline.html* to the list of *FILES_TO_CACHE*.
2. Handle the install event to pre-cache the files in *FILES_TO_CACHE*.
3. On activation, check and remove the old cached files in case our cache files and cache name have changed to include newer content.
4. Register yourself as the controller for all pages already created in this scope.
5. Handle the fetch event so that non-navigate events can be given back to the browser.
6. Check for failed navigate events and ensure the cached file can be served out of the local cache, in this case, local *offline.html* content.

```
// CODELAB: Update cache names any time
any of the cached files change.
const CACHE_NAME = "static-cache-v2";

// CODELAB: Add list of files to cache
here.
const FILES_TO_CACHE = ["/offline.
html"];

self.addEventListener("install", evt
=> {
  console.log("[ServiceWorker]
Install");

  // CODELAB: Precache static resources
here.
  evt.waitUntil(
    caches.open(CACHE_NAME).then(cache
=> {
      console.log("[ServiceWorker] Pre-
caching offline page");
      return cache.addAll(FILES_TO_
CACHE);
```

```
    })
  );
  self.skipWaiting();
});

self.addEventListener("activate", evt
=> {
  console.log("[ServiceWorker]
Activate");

  // CODELAB: Remove previous cached
data from disk.
  evt.waitUntil(
    caches.keys().then(keyList => {
      return Promise.all(
        keyList.map(key => {
          if (key !== CACHE_NAME) {
            console.
log("[ServiceWorker] Removing old
cache", key);
            return caches.delete(key);
          }
        })
      );
    })
  );

  self.clients.claim();
});

self.addEventListener("fetch", evt => {
  console.log("[ServiceWorker] Fetch",
evt.request.url);

  // CODELAB: Add fetch event handler
here.
  if (evt.request.mode !== "navigate")
  {

    // Not a page navigation, bail.
    return;
  }
  evt.respondWith(
    fetch(evt.request).catch(() => {
      return caches.open(CACHE_NAME).
then(cache => {
        return cache.match("offline.
html");
      });
    })
  );
});
```